

Strategic Architecture and Engineering Specification for "heatt": A Next-Generation Microblogging and Long-Form Hybrid Platform

1. Executive Summary and Strategic Vision

The contemporary digital ecosystem is increasingly defined by rigid demarcations between content modalities. Users are often forced to oscillate between the ephemeral, high-velocity interactions of microblogging networks and the deep, reflective engagement demanded by long-form publishing platforms. The architectural and design proposition for the application "heatt" seeks to actively dismantle this binary paradigm. By engineering a unified, seamless environment that natively harmonizes short-form updates with comprehensive long-form articles, the platform caters simultaneously to immediate social connectivity and sustained intellectual exploration.

Constructing a social platform from the ground up introduces the formidable "cold start" problem, wherein the absence of an initial user base results in a desolate interface devoid of engagement. To circumvent this without relying on artificial user generation or significant capital expenditure, the architecture leverages external, open-access API integrations to natively syndicate high-fidelity articles directly into the application feed. This approach guarantees immediate utility for early adopters. Furthermore, "heatt" distinguishes itself through an aggressively modern, visually immersive user experience characterized by a cinematic WebGL-driven onboarding sequence, sophisticated fluid typography, and psychologically rewarding gamification mechanisms such as an interactive, expanding activity heatmap. This report provides an exhaustive, ground-up blueprint encompassing both the aesthetic design system and the highly optimized, zero-cost engineering infrastructure required to deploy "heatt" at scale. It introduces a proprietary, physics-inspired "Heat Diffusion" ranking algorithm designed to surface trending content, alongside advanced edge-computing strategies for rendering rich link previews and optimizing media delivery.

2. Competitor Analysis and Market Positioning

To ensure "heatt" captures market share, it is imperative to analyze the structural deficiencies of existing platforms. The digital landscape reveals a distinct void that a hybrid platform can occupy by addressing the failures of both microblogging giants and dedicated publishing houses.

2.1 The Friction of Segregated Platforms

Traditional microblogging platforms excel at real-time information dissemination but structurally discourage nuanced discourse. Conversely, dedicated publishing platforms provide excellent reading experiences but suffer from high friction regarding content discovery and casual

interaction. Attempts to bridge this gap retroactively have historically struggled. For instance, the deployment of integrated long-form features on established microblogging platforms often suffers from poor user adoption because the core user base is conditioned for rapid consumption, and the algorithmic distribution heavily favors short-form outrage or virality over sustained reading.

2.2 The Decentralized Landscape

Emerging protocols such as ActivityPub and the AT Protocol present a different competitive vector. While these protocols decentralize identity and content ownership, their reference implementations frequently replicate the exact user experience of legacy microblogging platforms without innovating on the content modality itself. "heatt" positions itself uniquely by focusing primarily on the hybridization of content types and an unparalleled visual experience. By providing a medium where a 280-character observation can seamlessly coexist with a 2,000-word technical deep-dive—rendered natively without external redirects—the platform creates a continuous engagement loop that competitors lack.

Platform Type	Core Modality	Discovery Mechanism	Primary Deficiency	"heatt" Advantage
Legacy Microblogging	Short-form text, media	Algorithmic, high-velocity	Hostile to nuanced, long-form content	Native, distraction-free long-form integration
Publishing Platforms	Long-form articles	SEO, internal curation	High friction for casual, daily engagement	Seamless blending with rapid social updates
Decentralized Networks	Protocol-dependent	Federated timelines	Fragmentation and steep learning curves	Centralized, highly curated UX with potential future protocol sync

3. Aesthetic Paradigm and UI/UX Design System

The visual identity of "heatt" is engineered to evoke themes of thermal energy, intensity, and modern technological sophistication. The design system rejects the clinical, overly bright aesthetics typical of legacy social networks in favor of a "dark tech" paradigm that reduces ocular strain during long-form reading while allowing accent colors to provide striking visual feedback.

3.1 The "Dark Tech" Visual Identity and References

The foundation of the user interface utilizes deep blacks and charcoal greys, specifically avoiding absolute black (#000000) in favor of elevated dark tones (e.g., #0A0A0A to #111111) to preserve depth and shadow definition. The accent palette relies on luminescent gradients that simulate heat and fire, transitioning from deep indigo to vibrant teal, or from dark crimson to bright orange.

To visualize this aesthetic, the following conceptual references are central to the design inspiration:

- **Atmospheric Backgrounds:** The underlying theme is represented by abstract, glowing

gradients that simulate thermal energy dissipating in the dark. A primary visual reference is the concept of blurred flames glowing in darkness, which provides a dynamic yet non-distracting backdrop for the application shell.

- **High-Contrast Interface Elements:** UI components such as cards and buttons leverage subtle borders with glowing hover states, reminiscent of neon lighting against a dark gradient background. This creates a sense of depth and interactivity.

3.2 Fluid Typography and the Reading Experience

Because "heatt" is fundamentally a reading application, typographic excellence is paramount. The platform implements a fluid typography system utilizing modern CSS sizing functions to ensure perfect legibility across an infinite spectrum of device viewports.

Rather than relying on disjointed media query breakpoints that cause the layout to snap abruptly, the typography scales linearly. The CSS clamp() function dynamically calculates the optimal font size based on the viewport width, bounded by a defined minimum and maximum size. This ensures that an article heading maintains its hierarchical dominance whether viewed on a compact mobile screen or an ultrawide desktop monitor. The reading environment itself mimics the clean, distraction-free layout popularized by premium publishing platforms, utilizing a constrained single-column structure to maintain an optimal character-per-line count (typically between 65 and 75 characters).

4. The Cinematic Onboarding Experience

User retention drops precipitously during the initial seconds of application usage if the interface fails to captivate. To immediately establish "heatt" as a premium, differentiated product, the landing page and onboarding flow feature a cinematic, visually immersive sequence powered by real-time WebGL rendering.

4.1 Engineering the WebGL Heat Animation

The defining feature of the landing page is an interactive, fluid "heat" animation. This effect is achieved through custom GLSL (OpenGL Shading Language) shaders executed on the GPU, ensuring the animation operates at a flawless sixty frames per second without consuming significant CPU resources.

The architecture leverages React Three Fiber to instantiate a high-density plane geometry. A vertex shader manipulates the coordinates of this geometry using overlapping sinusoidal waves, creating a rippling, turbulent displacement that mimics the refractive distortion of heat haze. A uniform variable representing time (uTime) is continuously passed to the shader, driving the endless, organic motion of the waves.

The fragment shader subsequently maps these height variations to the platform's thermal color palette. Peaks in the geometry are assigned high-energy colors (e.g., cyan or bright orange), while the troughs fade into the dark background colors. To make the experience truly cinematic, the shader logic includes uniform variables bound to the user's cursor position or device gyroscope. As the user interacts with the screen, the focal point of the heat distortion dynamically shifts, creating a highly responsive and tactile introductory experience.

4.2 Performance Optimization Strategies

Deploying WebGL on the web necessitates rigorous performance optimization to prevent battery drain and thermal throttling on mobile devices. The application implements an adaptive performance degradation strategy. Upon initialization, a lightweight script profiles the device's GPU capabilities. If the hardware is constrained, the application seamlessly downgrades the dense WebGL geometry to a lower-polygon variant, or gracefully falls back to a highly optimized, CSS-driven gradient animation powered by libraries utilizing the Web Animations API to maintain hardware acceleration.

5. The Profile Architecture: Gamification via Heatmaps

A critical differentiating feature designed to drive daily active usage is the profile architecture. Drawing direct inspiration from the contribution graphs utilized by developer platforms, the user profile is anchored by a dynamic, interactive heatmap.

5.1 Psychological Mechanics of the Heatmap

The heatmap operates on established psychological principles of operant conditioning and visual progression. In standard social networks, a user's activity history is buried in an endless vertical timeline. The heatmap condenses this history into a highly visible, instantly readable grid. Each cell represents a day, and the color intensity (the "heat") correlates directly with the volume of the user's activity—such as articles published, posts authored, or deep reads completed.

This visualization inherently motivates users to maintain their "streak" and keep their grid illuminated. The fear of breaking a visual chain of activity serves as a powerful retention mechanism, transforming the passive act of scrolling into an active, gamified pursuit of consistency.

5.2 UI States: From Widget to Dashboard

The implementation of the heatmap card requires sophisticated interaction design to balance information density with aesthetic minimalism.

- **The Collapsed State:** By default, visiting a user profile reveals a compact, square widget. This element displays a localized, high-level summary of recent thermal output, presenting a visually appealing grid without overwhelming the profile layout.
- **The Expanded State:** When interacted with, the square card undergoes a fluid, layout-driven expansion (facilitated by libraries such as Framer Motion) to dominate the viewport. This expanded dashboard reveals the comprehensive annual grid.
- **Interactive Timelines and Metrics:** Within the expanded view, the interface provides granular data. Interacting with specific, high-intensity cells triggers the display of detailed timelines and rewarding metrics. The system dynamically generates narrative summaries of the user's achievements for that day, highlighting saved articles, the exact time of viral posts, and overall platform contribution, making the data feel personal and celebratory.

6. Solving the Cold Start: Algorithmic Content

Syndication

The most significant barrier to entry for any new social platform is the absence of an initial content ecosystem. A beautiful interface is meaningless if the feed is empty. To resolve this without financial expenditure, "heatt" implements a zero-cost syndication engine that populates the platform with high-quality, free articles relevant to the target demographic (e.g., technology, design, engineering).

6.1 Native Integration via the Forem (Dev.to) API

The platform achieves this by interfacing with the public REST API provided by Forem, the engine behind developer communities such as Dev.to. This API allows for the unauthenticated retrieval of published articles, returning comprehensive JSON payloads that include the author's metadata, engagement statistics, and, crucially, the raw Markdown body of the content. The architectural workflow for content ingestion is entirely server-side to ensure optimal client performance. A scheduled background process queries endpoints such as <https://dev.to/api/articles>, utilizing query parameters to filter by specific tags ensuring a diverse curation of subjects.

6.2 The Markdown Rendering Engine

A strict requirement of the platform is that all articles must be readable natively within the application; external redirects break the user experience and degrade the platform's value proposition. To accomplish this, the raw Markdown retrieved from the API must be parsed and transformed into the bespoke "heatt" visual style.

The application utilizes a robust parsing pipeline built upon the Unified ecosystem. The process begins with remark to parse the Markdown into an abstract syntax tree. This tree is then processed by rehype to convert the structure into HTML. Advanced plugins, such as rehype-highlight, are injected into the pipeline to provide sophisticated syntax highlighting for code blocks, a necessity for technical content.

Because this transformation occurs on the server during the static generation phase, the client receives pure, highly optimized HTML. This HTML is styled using the platform's fluid typography and dark aesthetic variables, ensuring that syndicated content feels entirely native to "heatt," complete with seamlessly integrated images and internal navigation links.

7. Edge-Rendered Open Graph Link Previews

A foundational feature of any microblogging application is the ability to automatically generate rich visual previews when a user includes a URL in their post. These previews typically require extracting Open Graph (OG) metadata (title, description, and high-resolution images) from the target website. While commercial APIs exist for this purpose, their free tiers are too restrictive for a scaling platform.

7.1 Streaming HTML Parsing at the Edge

To maintain the zero-cost mandate, "heatt" implements a proprietary link preview engine deployed as a serverless Edge Function. Traditional web scraping involves downloading the

entire HTML document of a target URL into memory, which is computationally expensive and slow. The optimized architecture utilizes a streaming HTML DOM parser. When a URL is submitted, the Edge Function initiates a fetch request to the target. Instead of waiting for the full response, it processes the incoming byte stream sequentially. The parser is configured to evaluate only the `<head>` section of the document, extracting the specific `meta[property="og:image"]`, `og:title`, and `twitter:card` tags. The critical optimization occurs immediately after the closing `</head>` tag is detected: the Edge Function forcibly terminates the network connection, aborting the download of the `<body>`. This drastically reduces bandwidth consumption, minimizes memory overhead, and returns the preview data to the client in milliseconds.

8. The "Heat Diffusion" Ranking Algorithm

Chronological feeds often fail to surface the most engaging content, while standard engagement-based algorithms (simple weighted sums of likes and retweets) are easily manipulated and prone to staleness. To ensure the feed remains dynamic, highly relevant, and mathematically robust, "heatt" employs a proprietary ranking model termed the "Heat Diffusion" algorithm, grounded in principles of thermodynamics and graph theory.

8.1 Mathematical Foundation and Thermal Mass

The algorithm models the platform as a complex bipartite graph connecting users and content nodes. Engagement actions (reading an article, expanding a heatmap, sharing a post) act as thermal conductors transferring "heat" across the network. The evolution of a post's trending score is governed by a discrete approximation of the heat equation, where the Laplacian operator calculates the flow of engagement based on the historical "thermal mass" (reputation and past engagement velocity) of the interacting users.

When a highly reputable user interacts with a post, they inject a significant spike of temperature into that node, initiating a diffusion process that propagates the content to adjacent user feeds.

8.2 Thermal Decay and Semantic Cliff Detection

To prevent older, highly upvoted content from permanently dominating the feed, the algorithm incorporates a strict thermal decay function. Based on Newton's Law of Cooling, the temperature of every post decays exponentially over time. Without a continuous, accelerating influx of new interactions to offset the decay, the content rapidly cools and descends in the rankings.

Furthermore, the algorithm employs advanced semantic cliff detection to dynamically truncate the visibility of content. By analyzing the local geometric structure of the sorted engagement logits via a position-weighted relative decay mechanism, the system identifies the exact moment a post's engagement velocity drops precipitously (the cliff). Content that falls off this cliff is immediately demoted, ensuring the feed constantly surfaces only the most actively "burning" discussions.

9. Zero-Cost Infrastructure and Edge Architecture

The financial mandate dictates that the entire platform must be architected using the free tiers of

modern cloud providers. This requires meticulous orchestration of compute resources, database operations, and content delivery networks.

9.1 Next.js and Vercel Deployment

The application is built using the Next.js App Router framework and deployed on Vercel. Vercel's free tier provides 100GB of bandwidth, unlimited static edge caching, and a generous allowance for serverless function execution. By strategically utilizing Incremental Static Regeneration (ISR) for the syndicated articles and landing pages, the compute load is drastically minimized. The pages are generated once on the server and then served from Vercel's global CDN, consuming zero compute resources on subsequent requests. Edge Functions, which lack the cold-start latency of traditional serverless functions, handle lightweight, high-frequency operations such as the aforementioned link preview extraction.

9.2 Aggressive Edge Caching via Stale-While-Revalidate

To protect the backend database from exhaustion and ensure the platform scales efficiently, the architecture relies heavily on the stale-while-revalidate (SWR) HTTP caching directive. When dynamic data is requested, the response headers are configured with Cache-Control: s-maxage=600, stale-while-revalidate=86400. This instructs the CDN edge nodes to immediately serve a slightly stale version of the feed to the user (resulting in single-digit millisecond response times) while simultaneously triggering an asynchronous, background revalidation request to the origin server to fetch the updated data. This completely removes the database query latency from the user's critical rendering path and consolidates thousands of concurrent requests into a single origin fetch, ensuring the free tier limits are never breached.

10. Database Schema: Relational vs. JSONB Optimization

Supabase provides the PostgreSQL backend for "heatt," offering a free tier that includes 500MB of storage and robust connection pooling. Designing the schema for a hybrid microblogging and long-form platform requires careful consideration of read performance versus structural flexibility.

While PostgreSQL's JSONB data type is highly versatile, utilizing it to store the core attributes of a high-velocity feed introduces severe performance penalties known as the "TOAST tax." When a JSONB payload exceeds 8KB, PostgreSQL moves the data to an out-of-line TOAST table. Subsequent sequential scans required to sort a feed force the database to perform costly decompression operations on every row, leading to exponential latency degradation as the table grows.

To optimize for maximum throughput, the schema employs a strict hybrid approach:

Data Entity	Storage Strategy	Architectural Reasoning
Users	Strict Relational	Requires rapid indexing for authentication, relationship mapping, and thermal mass calculations.

Data Entity	Storage Strategy	Architectural Reasoning
Posts (Micro)	Strict Relational	Feed generation demands high-velocity sequential scans and timestamp sorting; B-Tree indexes on relational columns avoid TOAST decompression overhead.
Articles (Long)	JSONB Payload	Long-form content comprises heterogeneous blocks (paragraphs, images, syntax-highlighted code). The structural flexibility of JSONB is necessary here, and single-record retrieval for reading does not suffer from sequential scan penalties.
Heatmap_Logs	Aggregated Relational	Daily interaction counts are aggregated into relational rows to rapidly query and render the 365-day GitHub-style contribution graph upon profile load.

11. Advanced Features for Virality and Platform Expansion

To fulfill the mandate of making the application "fun and cool to show off," the architecture includes extensible pathways for advanced features that increase the platform's utility and distribution potential.

11.1 Decentralized Protocol Synchronization

Recognizing the shift towards decentralized social graphs, the platform's architecture is designed to eventually support bridging to external networks. By leveraging protocols such as ActivityPub (the foundation of the Fediverse/Mastodon) and the AT Protocol (the foundation of Bluesky), "heatt" can act as a powerful centralized client for decentralized distribution. Users will be able to synchronize their long-form articles published on "heatt" directly to their AT Protocol or ActivityPub identities. This allows creators to utilize the superior aesthetic and typographic environment of "heatt" while seamlessly distributing their content to established audiences on other networks, driving inbound traffic back to the platform through rich link previews.

11.2 On-the-Fly Image Transformation

To ensure that the syndicated articles load instantly on mobile devices, the platform will utilize Cloudflare Workers to proxy and optimize external images. The free tier of Cloudflare provides sophisticated image transformation capabilities. By routing external image requests through a

dedicated Worker, the heavy, unoptimized image assets sourced from the Dev.to API are automatically resized, compressed, and converted into modern, efficient formats like WebP or AVIF before being cached and delivered to the client, further reducing bandwidth consumption and improving perceived performance.

12. Strategic Conclusions

The engineering and design specification for "heatt" demonstrates that a zero-budget constraint is not a barrier to creating a premium, highly complex digital platform, but rather a catalyst for architectural innovation. By strategically orchestrating the free tiers of Vercel, Supabase, and Cloudflare, and by leveraging public APIs to definitively solve the content cold-start problem, the platform is positioned for immediate, sustainable growth.

The hybridization of microblogging and long-form reading, wrapped in a deeply immersive "dark tech" visual identity and powered by cinematic WebGL rendering, provides an unparalleled user experience. Furthermore, the integration of advanced mathematical models—specifically the Heat Diffusion ranking algorithm and its semantic cliff detection—ensures that the platform's content discovery mechanism remains organic, dynamic, and highly engaging. This blueprint establishes "heatt" not merely as a functional application, but as a sophisticated, culturally relevant platform engineered for the modern social landscape.

Works cited

1. Microblogging platform Twitter has lost its status as an intermediary, <https://www.facebook.com/timesofoman/posts/microblogging-platform-twitter-has-lost-its-status-as-an-intermediary-platform-i/4128364840576473/> 2. Report: Twitter Working on Long-Form Content Support Known as, <https://winbuzzer.com/2022/06/22/report-twitter-working-on-long-form-content-support-known-as-twitter-notes-xcxwbn/> 3. GitHub-style Habit Tracker - HabitHeat.com, <https://habitheat.com/github-style-habit-tracker/> 4. AT Protocol - Wikipedia, https://en.wikipedia.org/wiki/AT_Protocol 5. (PDF) Looking AT the Blue Skies of Bluesky - ResearchGate, https://www.researchgate.net/publication/383308458_Looking_AT_the_Blue_Skies_of_Bluesky 6. Dark Gradient Pictures | Download Free Images on Unsplash, <https://unsplash.com/s/photos/dark-gradient> 7. Unsplash Brand Assets and Guidelines, <https://unsplash.com/press> 8. Abstract blurred flames glowing in darkness - Free Photo on Unsplash, <https://unsplash.com/photos/abstract-blurred-flames-glowing-in-darkness-9f5T2o93XSA> 9. Abstract light waves glow against a dark background - Unsplash, <https://unsplash.com/photos/abstract-light-waves-glow-against-a-dark-background-GCjb21XeHk> 10. Fluid typography with CSS clamp - Piccalilli, <https://piccalil.li/blog/fluid-typography-with-css-clamp/> 11. We showcase top blog design layouts (and how to build them), <https://hyvor.com/blog/blog-design> 12. An introduction to shaders | Articles - web.dev, <https://web.dev/articles/webgl-shaders> 13. Animated Heat Distortion Effects with WebGL - Codrops, <https://tympanus.net/codrops/2016/05/03/animated-heat-distortion-effects-webgl/> 14. WebGL Performance - Wonderland Engine, <https://wonderlandengine.com/about/webgl-performance/> 15. Write a Wave Shader — Three.js lesson - DevTools daily,

<https://www.devtoolsdaily.com/three/examples/shader-waves/> 16. Animation performance guide - Motion.dev, <https://motion.dev/docs/performance> 17. An Interactive Guide to Web Animation Performance, <https://www.siwalik.in/posts/interactive-guide-to-web-animation-performance/> 18. GitHub-style habit tracker: why it works - init.Habits, <https://inithabits.com/blog/github-style-habit-tracker> 19. Build a Blog using Next.JS and DEV.to, <https://dev.to/martinp/build-a-blog-using-nextjs-and-devto-15a5> 20. How to use DEV.to API, <https://dev.to/guilhermecheng/how-to-use-devto-api-4p65> 21. DEV API (beta) - Introduction | Forem Docs, <https://developers.forem.com/api/v0> 22. How to use DEV.to API (2025), <https://dev.to/ketanchavan/devto-api-fetch-public-private-data-effortlessly-1nba> 23. How to use the Remark Markdown converters with Next.js projects, <https://dev.to/jameswallis/how-to-use-the-remark-markdown-converters-with-next-js-projects-a8a24> 24. how to extract open graph metadata playwright - GitHub, https://github.com/feder-cr/invisible_playwright/wiki/how-to-extract-open-graph-metadata-playwright 25. How to Extract Metadata from Web Pages (Open Graph, Schema.org), <https://fast.io/resources/metadata-extraction-from-web-pages-open-graph/> 26. Website Logo API - Logos, Favicons & Brand Colors - Microlink, <https://microlink.io/logo> 27. StreamParser: A hybrid Java SAX + DOM parser for large documents, <https://jsoup.org/cookbook/input/streamparser-dom-sax> 28. Extract links and metadata from HTML - Deno Docs, https://docs.deno.com/examples/extract_html_data/ 29. Extract social share images and Open Graph tags | Bun Guides, <https://bun.com/guides/html-rewriter/extract-social-meta> 30. A Consistent Diffusion-Based Algorithm for Semi-Supervised ... - arXiv, <https://arxiv.org/html/2008.11944v1> 31. An Operator-Based Approach for Modeling Influence Diffusion in, https://www.researchgate.net/publication/354084303_An_Operator-Based_Approach_for_Modeling_Influence_Diffusion_in_Complex_Social_Networks 32. Inferring users' preferences through leveraging their social ... - arXiv, <https://arxiv.org/pdf/1711.10399> 33. Scalable Similarity Search over Large Attributed Bipartite Graphs, https://www.researchgate.net/publication/398675276_Scalable_Similarity_Search_over_Large_Attributed_Bipartite_Graphs 34. Generator-based Graph Generation via Heat Diffusion - arXiv, <https://arxiv.org/pdf/2602.03612> 35. A Heat-Map-based Algorithm for Recognizing Group Activities in, <https://arxiv.org/pdf/1502.06076> 36. Min-k Sampling: Decoupling Truncation from Temperature Scaling, <https://arxiv.org/html/2604.11012v1> 37. evgenygurin/vercel-mcp-memory - GitHub, <https://github.com/evgenygurin/vercel-mcp-memory> 38. free-for-dev/README.md at master - GitHub, <https://github.com/ripienaar/free-for-dev/blob/master/README.md?plain=1> 39. Edge Functions | Supabase Docs, <https://supabase.com/docs/guides/functions> 40. Understanding Stale-While-Revalidate: Serving Cached Content, <https://www.debugbear.com/docs/stale-while-revalidate> 41. Cache-Control and stale-while-revalidate for CDN Caching, <https://blog.blazingcdn.com/en-us/cache-control-stale-while-revalidate-cdn-caching> 42. DmitryScaletta/free-database-services - GitHub, <https://github.com/DmitryScaletta/free-database-services> 43. Postgres JSONB Columns and TOAST: A Performance Guide, <https://www.snowflake.com/en/blog/engineering/postgres-jsonb-columns-and-toast/> 44. JSONB vs Text in PostgreSQL: Which Performs Better?, <https://www.cybrosys.com/research-and-development/postgres/jsonb-vs-text-in-postgresql-which-performs-better> 45. Stop Posting to Platforms — Turn Your Website into a Fediverse Node, <https://ztfr.eu/stop-posting-to-platforms-turn-your-website-into-a-fediverse-node/> 46. Pricing · Cloudflare Images docs, <https://developers.cloudflare.com/images/pricing/> 47. Transform via

fetch · Cloudflare Images docs,
<https://developers.cloudflare.com/images/optimization/transformations/transform-via-workers/>
48. Proxying an Image CDN with Cloudflare workers - Alistair Shepherd,
<https://alistairshepherd.uk/writing/cloudflare-worker-image-proxy/>